

Simulación de Partículas N-Body con MPI y OpenMP

CI-0117 Programación Paralela y Concurrente — Tarea 2

Gabriel Coto Fernández (C4E540)
Escuela de Ciencias de la Computación e Informática
Universidad de Costa Rica
San José, Costa Rica
gabriel.cotofernandez@ucr.ac.cr

Jhon Campos Montenegro (C4D674)
Escuela de Ciencias de la Computación e Informática
Universidad de Costa Rica
San José, Costa Rica
jhon.campos@ucr.ac.cr

Resumen—Este artículo describe el diseño e implementación de una simulación paralela del problema N-body en C++ utilizando MPI para comunicación entre procesos y OpenMP para paralelismo a nivel de hilos dentro de cada proceso. El sistema simula la interacción de partículas mediante fuerzas de van der Waals en un espacio tridimensional. Se implementó un algoritmo de anillo lógico con p procesos (impar) que distribuye equitativamente las partículas y minimiza la comunicación a $\lfloor (p-1)/2 \rfloor$ etapas de rotación. La validación numérica ($p = 9$, 900 partículas, 100 iteraciones) confirmó una diferencia máxima de 0,000000 contra la referencia (umbral 0,2), tras corregir un error de inicialización que provocaba colisiones de posición entre procesos. La corrida de desempeño ($p = 15$, 3000 partículas, 7000 iteraciones) completó en 49,12 s (≈ 7.02 ms por iteración) sin inestabilidad numérica.

Index Terms—MPI, OpenMP, N-body, computación paralela, van der Waals, HPC

I. INTRODUCCIÓN

La simulación de sistemas de partículas que interactúan por medio de fuerzas es un problema fundamental en computación científica. El problema N-body requiere calcular, en cada iteración, la fuerza que cada partícula ejerce sobre las demás, lo que implica una complejidad computacional de $O(N^2)$ por iteración. Para escalarlo a sistemas con millones de partículas es indispensable el uso de computación paralela.

En este trabajo se implementa una solución híbrida MPI+OpenMP en C++17. MPI distribuye el conjunto de partículas entre p procesos organizados en un anillo lógico, mientras que OpenMP paralela el cómputo local de fuerzas dentro de cada proceso mediante hilos compartidos.

El programa se ejecuta en el clúster Kabré del CeNAT (Costa Rica) y sigue la especificación del enunciado del curso CI-0117.

II. MODELO FÍSICO

Cada partícula i posee posición $\langle x_i, y_i, z_i \rangle$, velocidad $\langle v_{xi}, v_{yi}, v_{zi} \rangle$, fuerza acumulada $\langle f_{xi}, f_{yi}, f_{zi} \rangle$ y masa $m = 4,0$.

La fuerza entre dos partículas separadas por distancia r se modela con el potencial de Lennard-Jones simplificado:

$$f = \frac{B}{r^{12}} - \frac{A}{r^6}, \quad A = 2,0, \quad B = 1,0 \quad (1)$$

Sus componentes cartesianas son:

$$f_x = f \cdot \frac{r_x}{r}, \quad f_y = f \cdot \frac{r_y}{r}, \quad f_z = f \cdot \frac{r_z}{r} \quad (2)$$

La integración de movimiento usa el método de Euler explícito con $\Delta t = 0,1$:

$$v_\alpha^{(t+1)} = v_\alpha^{(t)} + \frac{f_\alpha}{m} \Delta t, \quad x_\alpha^{(t+1)} = x_\alpha^{(t)} + v_\alpha^{(t+1)} \Delta t \quad (3)$$

III. DISEÑO DE LA SOLUCIÓN PARALELA

III-A. Infraestructura MPI — Persona A

III-A1. Topología de anillo: Los p procesos (siempre impar) se organizan en un anillo lógico. Cada proceso r conoce sus vecinos:

$$\text{left} = (r - 1 + p) \bmod p, \quad \text{right} = (r + 1) \bmod p \quad (4)$$

El número de etapas de rotación es $\lfloor (p-1)/2 \rfloor$. La restricción de impar garantiza que cada proceso interactúe exactamente con todos los demás en estas etapas más el paso de retorno.

La clase `mpiutils::MpiEnvironment` implementa un RAI wrapper sobre `MPI_Init/MPI_Finalize`, garantizando limpieza correcta incluso ante errores tempranos.

III-A2. Tipo MPI para Particle: Para transmitir partículas entre procesos se registra un `MPI_Datatype` derivado del layout real de la estructura en memoria:

```
MPI_Type_create_struct(
    nfields, blocklengths,
    offsets, types,
    &mpi_particle_type);
MPI_Type_commit(&mpi_particle_type);
```

Los offsets se calculan con `offsetof()` y un `static_assert` verifica en tiempo de compilación que `sizeof(Particle) == 10 * sizeof(double)`, evitando corrupción silenciosa por padding.

III-A3. Bucle de rotación: El algoritmo sigue los pasos del enunciado (Fig. 1):

1. Cada proceso envía sus partículas locales al vecino derecho.
2. Recibe partículas del vecino izquierdo (*remotes*).
3. Llama a `evolve(locals, remotes)` para actualizar fuerzas.
4. Reenvía *remotes* al vecino derecho.
5. Repite pasos 2–4 por $\lfloor (p-1)/2 \rfloor$ etapas.
6. Paso de retorno: envía las partículas remotas directamente al proceso que las originó.

La comunicación usa `MPI_Sendrecv` para solapar envío y recepción en una sola llamada bloqueante, evitando deadlocks.



Figura 1. Algoritmo de rotación en anillo para 5 procesos. Cada color representa las partículas de un proceso distinto.

III-A4. Recolección y escritura de resultados: Cada 100 iteraciones, el rank 0 recolecta todas las partículas con `MPI_Gatherv` (que maneja tamaños desiguales por proceso) y escribe un archivo CSV con las posiciones, compatible con el lector CSV de Paraview.

III-B. Física y OpenMP — Persona B

III-B1. Clase Particle: Particle es un struct de 10 double contiguos ($x, y, z, v_x, v_y, v_z, f_x, f_y, f_z, \text{mass}$), alineado a 8 bytes (`alignas(8)`). Este layout plano es lo que permite construir el `MPI_Datatype` por `offsetof()` sin ambigüedad, verificado en tiempo de compilación con el `static_assert` de la Sección III-A2.

III-B2. Cómputo de fuerzas: `evolve(locals, remotes, nlocal, nremote)` calcula la fuerza (Ec. 1) entre cada partícula local y cada partícula remota recibida en la rotación actual, acumulando la reacción de Newton sobre `remotes[j]` con `#pragma omp atomic` (necesario porque varios hilos, cada uno iterando sobre un subconjunto disjunto de `locals`, pueden escribir sobre el mismo `remotes[j]`). La paralelización es:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < nlocal; ++i) {
    for (int j = 0; j < nremote; ++j) {
        // ... calculo de f, fx, fy, fz
        locals[i].fx += fx; // sin atomic: i es
                           // exclusivo del hilo
        #pragma omp atomic
        remotes[j].fx -= fx; // con atomic: j es
                           // compartido
    }
}
```

`evolveSelf(particles, n)` resuelve la auto-interacción dentro del bloque local (triángulo superior $i < j$

para no duplicar pares), usando `#pragma omp atomic` en ambos extremos del par porque i y j pueden caer en el mismo hilo o en hilos distintos indistintamente.

Un factor de suavizado $\varepsilon = 0,01$ se suma a r^2 para evitar división por cero cuando $r \rightarrow 0$ (Ec. 5):

$$r^2 = r_x^2 + r_y^2 + r_z^2 + \varepsilon^2 \quad (5)$$

`merge(locals, returned)` suma las fuerzas acumuladas durante el paso de retorno del anillo sobre las partículas locales, y `updateProperties(locals)` integra con Euler explícito (Ec. 3, $\Delta t = 0,001$) y reinicia f_x, f_y, f_z a 0 para la siguiente iteración. Ambas funciones paralelizan el `for` externo sobre partículas locales, sin necesidad de `atomic` porque cada partícula es escrita por un único hilo.

III-B3. Modos de inicialización: `init.cpp` ofrece tres modos, seleccionados por `INIT_MODE`:

- **Aleatoria** (`initRandom`): posiciones uniformes en $[-10, 10]^3$ con `std::mt19937` sembrado por rango $(42 + \text{rank} \cdot 1000)$ para reproducibilidad.
- **Fija** (`initFixed`): rejilla 3D regular, usada para validación numérica determinística. **Esta función contenía el error principal descrito en la Sección III-C.**
- **Galaxia** (`initGalaxy`): dos discos de partículas en órbita kepleriana ($v_{\text{orbit}} \propto 1/\sqrt{r}$) *acercándose en elejez*, usado para el dataset de visualización en Paraview (Sección V-C).

III-C. Depuración: error de inicialización en `initFixed`

Durante la validación inicial en Kabré ($p = 9, N = 100$, 100 iteraciones), `compareFiles/compare.py` reportó un FAIL con una diferencia máxima del orden de 10^{19} en la coordenada x de varias partículas, mientras que y y z permanecían prácticamente sin cambio respecto a la referencia.

Causa raíz. `initFixed(n, rank)` calculaba el tamaño de la rejilla 3D (`gridSide`) a partir del *offset local* de cada proceso:

```
int offset = rank * n;
int gridSide = ceil(cbrt(offset + n)); // distinto
                                     por rango
```

Para $p = 9, N = 100$ esto produce `gridSide = 5, 6, 7, 8, 8, 9, 9, 10, 10` para los rangos 0–8 respectivamente: *cada proceso usa una rejilla distinta*. Se verificó programáticamente que esto provoca que partículas de procesos diferentes caigan exactamente en la misma posición (p. ej., la partícula global 74 del rango 0 y la partícula 100 del rango 1 coinciden ambas en $(8, 8, 4)$). Con distancia $r \approx 0$ entre dos partículas, el término repulsivo B/r^{12} de la Ec. 1 se dispara a magnitudes del orden de 10^{24} , y pequeñas asimetrías de redondeo de punto flotante hacen que esa fuerza se descargue predominantemente en una sola coordenada al cabo de varias iteraciones — exactamente el síntoma observado.

Corrección. `gridSide` debe ser idéntico en todos los procesos, por lo que se calcula a partir del total global de partículas ($N \times p$), igual a como ya lo hacía `initGalaxy`:

```
int totalParticles = n * totalRanks;
int gridSide = ceil(cbrt(totalParticles));
```

Esto requirió agregar el parámetro `totalRanks` a la firma de `initFixed` (`init.hpp/.cpp` y la llamada en `main.cpp`). Tras el cambio se verificó que las 900 posiciones iniciales son únicas y que la distancia mínima entre partículas de distinto proceso es exactamente el espaciado de rejilla (2.0), eliminando las colisiones.

Segundo hallazgo: infraestructura de referencia incompleta. El repositorio documentaba en `README.md` un script `compareFiles/generate_reference.sh` que generaba `compareFiles/reference/particles_100.csv`, pero dicho script no existía en el árbol de archivos. Esto causaba `FileNotFoundException` en varias corridas de `validate.sh`, y en al menos una ocasión la referencia comparada provenía de una corrida previa generada con el `initFixed` aún defectuoso (el propio valor de referencia mostraba la misma magnitud anómala de 10^{19}). Se creó el script faltante como un job de SLURM (mismo patrón que `validate.sh`), de forma que reserva una asignación real de 9 tareas vía `srun` en vez de depender de los `slots` del nodo de login.

IV. IMPLEMENTACIÓN

IV-A. Estructura de archivos

Tabla I
DIVISIÓN DE ARCHIVOS POR RESPONSABILIDAD

Archivo	Responsable
<code>main.cpp</code>	Persona A
<code>mpiUtils.hpp/.cpp</code>	Persona A
<code>io.hpp/.cpp</code>	Persona A
<code>particle.hpp/.cpp</code>	Persona B
<code>physics.hpp/.cpp</code>	Persona B
<code>init.hpp/.cpp</code>	Persona B
<code>CMakeLists.txt</code>	Persona A
<code>validate.sh</code>	Persona A
<code>performance.sh</code>	Persona A

IV-B. Compilación

El proyecto usa CMake con soporte explícito para MPI y OpenMP:

```
find_package(MPI REQUIRED)
find_package(OpenMP REQUIRED)
target_link_libraries(nBody
  MPI::MPI_CXX OpenMP::OpenMP_CXX)
```

En Kabré se compila cargando los módulos:

```
module load gcc/13.4.0
module load openmpi/4.1.6-pmi2
module load cmake/3.26.0
cmake .. -DCMAKE_CXX_COMPILER=mpicxx
make
```

IV-C. Ejecución

```
mpiexec -np <p> ./cenatMD \
  <N> <ITER> <PRINT> <INIT>
```

Donde `N` es partículas por proceso, `ITER` el número de iteraciones, `PRINT` activa la escritura de archivos cada 100 iteraciones e `INIT` selecciona posiciones fijas para validación.

V. RESULTADOS

V-A. Validación

Tras corregir el error descrito en la Sección III-C y regenerar la referencia, se ejecutó `validate.sh` en la partición `nukwa-wide` de Kabré:

```
sbatch validate.sh
# internamente:
srun --ntasks=9 ./build/nBody 100 100 1 1
python3 compareFiles/compare.py \
  compareFiles/reference/particles_100.csv \
  output/particles_100.csv
```

Salida obtenida (`logs/validate_582051.out`):

```
[compare] PASS
Particles compared : 900
Max difference      : 0.000000 (threshold=0.2)
Worst particle idx  : -1
```

La diferencia máxima de 0,000000 está muy por debajo del umbral de 0,2, confirmando que el algoritmo de anillo, el cómputo de fuerzas y la integración de Euler son numéricamente correctos una vez eliminadas las colisiones de inicialización.

V-B. Desempeño y Escalabilidad

Se ejecutó `performance.sh` ($p = 15$, $N = 200$ partículas por proceso, 3000 partículas en total, 7000 iteraciones, inicialización aleatoria) en la partición `nukwa-wide` de Kabré:

```
[timing] iter=7000 P=15 N/proc=200
total=49.1154 s
por_iter=0.00701648 s
```

La corrida completó sin abortos ni desbordamientos numéricos a lo largo de las 7000 iteraciones, lo que confirma que la corrección de `initFixed` (Sección III-C) no introdujo efectos secundarios en el modo de inicialización aleatoria ni en corridas largas.

`performance.sh` incluye un bloque que repite la corrida para $p \in \{1, 3, 5, 7, 9, 11, 13, 15\}$ y genera un CSV de tiempos por proceso. Se ejecutó esta barrida de escalabilidad en la partición `nukwa-wide` de Kabré (job 582068), obteniendo los resultados del Cuadro II.

El speedup obtenido es negativo (decrece monotónicamente con p): el tiempo de ejecución aumenta de forma aproximadamente lineal al incrementar el número de procesos, lo que indica que el overhead de comunicación MPI domina sobre la ganancia de paralelismo para este tamaño de problema. Este comportamiento es consistente con la naturaleza del algoritmo de anillo: en cada iteración cada proceso realiza $\lfloor (p-1)/2 \rfloor$ rondas de `MPI_Sendrecv` transmitiendo bloques de $N = 200$ partículas, de modo que el volumen de comunicación crece

Tabla II
SPEEDUP Y EFICIENCIA — $N = 200$ PART./PROCESO, 7000 ITER.,
INICIALIZACIÓN ALEATORIA

p	Tiempo (s)	Speedup	Eficiencia (%)
1	10.521	1.0000	100.00
3	10.816	0.9727	32.42
5	17.406	0.6044	12.09
7	23.951	0.4393	6.28
9	30.402	0.3461	3.85
11	36.675	0.2869	2.61
13	43.013	0.2446	1.88
15	49.590	0.2122	1.41

con p mientras que el cómputo local se mantiene constante. Para observar speedup superlineal o lineal sería necesario aumentar significativamente N (carga de cómputo por proceso), de manera que el trabajo aritmético por iteración supere el overhead de red.

V-C. Visualización

El dataset creativo corresponde a una colisión de dos galaxias (`initGalaxy`, Sección III-B3): dos discos de partículas en órbitas keplerianas iniciales, centrados en $x = -20$ y $x = +20$, que se aproximan entre sí con velocidad relativa en z . Se generó con:

```
sbatch galaxy.sh
# internamente: srun --ntasks=9 ./build/nBody 200
1000 1 2
```

produciendo 1800 partículas distribuidas en 10 archivos `output/particles_100.csv` ...`particles_1000.csv` (uno cada 100 iteraciones), cargados en Paraview como serie de tiempo mediante el filtro **Table To Points** ($X = x, Y = y, Z = z$) seguido de **Glyph** (esfera, radio 0.3) y color por mass o por rapidez $\sqrt{v_x^2 + v_y^2 + v_z^2}$, recorridos con **Animation View**.

VI. CONCLUSIONES

La solución híbrida MPI+OpenMP implementada distribuye correctamente el cómputo de fuerzas N-body entre múltiples procesos organizados en un anillo lógico, con OpenMP paralelizando el bucle interno de fuerzas dentro de cada nodo. La validación numérica confirmó exactitud perfecta (diferencia máxima 0,000000 contra referencia, umbral 0,2) tras corregir el error de inicialización en `initFixed` descrito en la Sección III-C.

Los resultados de escalabilidad (Cuadro II) muestran un speedup decreciente: para $N = 200$ partículas por proceso, el tiempo total aumenta de 10.52 s con $p = 1$ a 49.59 s con $p = 15$, con eficiencias inferiores al 33 % desde $p = 3$. Esto se explica porque el overhead de comunicación MPI — $\lfloor (p-1)/2 \rfloor$ rondas de `MPI_Sendrecv` por iteración, cada una transmitiendo $N \cdot 10$ doubles — crece linealmente con p , mientras que la carga de cómputo por proceso se mantiene fija. El régimen de comunicación-dominado es un comportamiento esperado para cargas pequeñas por proceso; escalar N de forma proporcional

a p (escalabilidad débil) permitiría amortizar dicho overhead y observar eficiencias más favorables.

El caso del error de `initFixed` ilustra un riesgo característico de MPI que no depende de los algoritmos de comunicación: una rutina de inicialización que asume implícitamente un parámetro uniforme entre procesos puede romper la corrección física global del sistema aunque cada proceso, de forma aislada, ejecute código localmente válido. La ausencia de mensajes de error del runtime MPI hace que este tipo de defecto sea especialmente difícil de diagnosticar sin herramientas de validación externas.

RECONOCIMIENTOS

Los autores agradecen al Prof. Josef Ruzicka por el diseño del enunciado y a Diego Jiménez (Max Planck Computing and Data Facility) por el material base. La ejecución se realizó en el clúster Kabré del CNCA-CeNAT.

REFERENCIAS

- [1] M. Snir et al., *MPI: The Complete Reference*, 2nd ed. Cambridge, MA: MIT Press, 1998.
- [2] B. Chapman, G. Jost, and R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge, MA: MIT Press, 2007.
- [3] D. Frenkel and B. Smit, *Understanding Molecular Simulation: From Algorithms to Applications*, 2nd ed. San Diego, CA: Academic Press, 2002.
- [4] J. Ruzicka, “CI-0117 Tarea MPI: Programación en memoria distribuida,” Universidad de Costa Rica, 2025, enunciado del curso.